Dynamic Programming Study Guide

1. What is Tabulation / Bottom Up Approach

Tabulation is a dynamic programming technique where we solve problems by building up solutions from the smallest subproblems to the final answer. This approach uses iteration and stores results in a table (usually an array) to avoid recalculating the same subproblems.

Key Features:

- Uses iteration instead of recursion
- Builds solutions from bottom to top
- Stores results in a table/array
- No function call overhead
- Generally more space and time efficient

How Tabulation Works:

1. **Identify the base cases** - These are the smallest subproblems
2. **Create a table** - Usually an array to store results
3. **Fill the table iteratively** - Start from base cases and work up
4. **Return the final result** - Usually the last element or a specific index

Example: Fibonacci Sequence using Tabulation

```java
public class FibonacciTabulation {
    public static int fibonacci(int n) {
        if (n <= 1) {
            return n;
        }

        // Create table to store results
        int[] dp = new int[n + 1];

        // Base cases
        dp[0] = 0;
        dp[1] = 1;

        // Fill table from bottom up
        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }

    public static void main(String[] args) {
        System.out.println("Fibonacci(10) = " + fibonacci(10)); // Output: 55
    }
}
```

Example: Climbing Stairs using Tabulation

```java
public class ClimbingStairs {
    public static int climbStairs(int n) {
        if (n <= 2) {
            return n;
        }

        int[] dp = new int[n + 1];
        dp[1] = 1; // 1 way to climb 1 step
        dp[2] = 2; // 2 ways to climb 2 steps

        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }

    public static void main(String[] args) {
        System.out.println("Ways to climb 5 stairs: " + climbStairs(5)); // Output: 8
    }
}
```

2. What is Memoization / Top Down Approach

Memoization is a dynamic programming technique that uses recursion with caching to store the results of expensive function calls. When the same subproblem is encountered again, we return the cached result instead of recalculating it.

Key Features:

- Uses recursion with caching
- Solves problems from top to bottom
- Stores results in a cache (HashMap, array, etc.)
- Only calculates subproblems that are actually needed
- Natural recursive approach

How Memoization Works:

1. **Check if result exists** - Look in the cache first
2. **Return cached result** - If found, return it immediately

3. **Calculate and store** - If not found, calculate the result and store it
4. **Recursive calls** - Make recursive calls for subproblems

Example: Fibonacci Sequence using Memoization

```java
import java.util.HashMap;
import java.util.Map;

public class FibonacciMemoization {
    private static Map<Integer, Integer> memo = new HashMap<>();

    public static int fibonacci(int n) {
        // Base cases
        if (n <= 1) {
            return n;
        }

        // Check if result is already computed
        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        // Calculate and store result
        int result = fibonacci(n - 1) + fibonacci(n - 2);
        memo.put(n, result);

        return result;
    }

    public static void main(String[] args) {
        System.out.println("Fibonacci(10) = " + fibonacci(10)); // Output: 55
    }
}
```

Example: Climbing Stairs using Memoization

```java
import java.util.HashMap;
import java.util.Map;

public class ClimbingStairsMemo {
    private static Map<Integer, Integer> memo = new HashMap<>();

    public static int climbStairs(int n) {
        // Base cases
        if (n <= 2) {
            return n;
        }

        // Check cache
        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        // Calculate and store
        int result = climbStairs(n - 1) + climbStairs(n - 2);
        memo.put(n, result);

        return result;
    }

    public static void main(String[] args) {
        System.out.println("Ways to climb 5 stairs: " + climbStairs(5)); // Output: 8
    }
}
```

Comparison: Tabulation vs Memoization

Aspect	Tabulation (Bottom-Up)	Memoization (Top-Down)
Approach	Iterative	Recursive
Memory Usage	Fixed table size	Dynamic cache
Function Calls	No recursion overhead	Recursive calls
Space Complexity	O(n) table space	O(n) cache + O(n) call stack
Ease of Implementation	Requires careful order	Natural recursive flow

---

Quiz Questions

Question 1: What is the main difference between tabulation and memoization? a) Tabulation uses recursion, memoization uses iteration b) Tabulation uses iteration, memoization uses recursion c) Both use recursion d) Both use iteration

Answer: b) Tabulation uses iteration, memoization uses recursion

Question 2: In the tabulation approach, how do we typically store the results? a) HashMap b) Array/Table c) Stack d) Queue

Answer: b) Array/Table

Question 3: What happens when we encounter a subproblem we've already solved in memoization? a) We solve it again b) We return the cached result c) We skip it entirely d) We throw an exception

Answer: b) We return the cached result

Question 4: Which approach generally has less space complexity? a) Tabulation b) Memoization c) Both are equal d) Depends on the problem

Answer: a) Tabulation

Question 5: In memoization, what do we check first before calculating a result? a) Base cases b) If the result is in cache c) The input parameters d) The recursive formula

Answer: b) If the result is in cache

Practice Coding Questions

Question 1: Implement a function to calculate the minimum number of coins needed to make a given amount using tabulation approach. You have coins of denominations [1, 5, 10, 25].

Question 2: Write a memoized solution for calculating the number of unique paths in a grid from top-left to bottom-right, where you can only move right or down.

Question 3: Implement both tabulation and memoization approaches for the "House Robber" problem: Given an array of integers representing money in each house, find the maximum amount you can rob without robbing two adjacent houses.

Question 4: Create a memoized function to solve the "Longest Common Subsequence" problem between two strings.

Question 5: Write a tabulation solution for the "0/1 Knapsack" problem where you need to maximize value within a weight constraint.